



John the Ripper: A Comprehensive Analysis of the Password Security Auditing Tool

John the Ripper: A Comprehensive Analysis of the Password Security Auditing Tool

Course: Cybersecurity

Submission Date: December 8, 2025

Group Members

1. Hanan Hamza
2. Abdurrehman Awan
3. Ehtisham Jalil
4. Ali Khan
5. Abdul Rehman

Table of Contents

Introduction

- 1.1. What is John the Ripper?**
- 1.2. The Openwall Project and Solar Designer**
- 1.3. History and Evolution**
- 1.4. Importance in Modern Cybersecurity**
- 1.5. The Origin of the Name**

Types of Attacks in John the Ripper

- 2.1. Wordlist Attack**
- 2.2. Brute Force Attack (Incremental Mode)**
- 2.3. Single Crack Attack**
- 2.4. Mask Attack**

Conclusion

1. Introduction



1.1. What is John the Ripper?

John the Ripper (JtR) is a powerful, free, and open-source password security auditing and password recovery tool . It is designed to detect weak passwords in a system by performing rapid, automated password guessing against encrypted password hashes. Originally developed for the Unix operating system, JtR has since been ported to run on over fifteen different platforms, including macOS, Windows, and various Linux distributions, making it a highly versatile tool in the cybersecurity professional's arsenal [2].

The core function of JtR is to take a list of password hashes (the encrypted representations of passwords) and attempt to "crack" them by generating millions of potential passwords, hashing them, and comparing the resulting hash with the target hash. This process allows system administrators and security analysts to identify user accounts with easily guessable passwords, thereby strengthening overall system security [3].

1.2. The Openwall Project and Solar Designer

John the Ripper was created by **Alexander Peslyak**, who is more widely known by his pseudonym, **Solar Designer**. Solar Designer is a respected figure in the information security community and is the founder of the **Openwall Project**. The Openwall Project is a community-driven initiative focused on security-related software development and vulnerability research, with JtR being one of its most prominent and successful contributions .The project emphasizes the development of robust, secure, and open-source solutions for various operating systems.

1.3. History and Evolution

The development of John the Ripper began in the **late 1990s**, a period when password security was becoming a critical concern for Unix-based systems. Its initial purpose was to consolidate multiple existing password cracking tools into a single, unified, and highly optimized package.

Era	Key Development/Feature	Significance
Late 1990s	Initial release, focus on Unix <u>crypt(3)</u> hashes.	Established JtR as the leading tool for Unix password auditing.
Early 2000s	Introduction of the "Jumbo" community-enhanced version.	Expanded support to hundreds of hash types (e.g., NTLM, MySQL, LDAP) and ciphers.
2010s - Present	GPU acceleration, OpenMP parallelization, and advanced modes.	Significantly increased cracking speed, allowing for the auditing of modern, complex hashes like SHA-crypt and bcrypt [7].

The tool's continuous evolution, particularly the development of the "Jumbo" branch, has ensured its relevance by adapting to new hashing algorithms and leveraging modern hardware capabilities, such as multi-core CPUs and Graphics Processing Units (GPUs) for parallel processing [7].

1.4. Importance in Modern Cybersecurity

John the Ripper is an indispensable tool across several domains of modern cybersecurity:

- **Penetration Testing:** Ethical hackers use JtR to simulate real-world attacks, identifying weak passwords that could serve as an entry point for malicious actors.
- **Password Auditing:** System administrators regularly employ JtR to audit their user base, ensuring compliance with strong password policies and mitigating the risk of credential-based breaches.
- **Digital Forensics:** In forensic investigations, JtR can be used to recover passwords from seized devices or encrypted files, aiding in the analysis of digital evidence.
- **Security Analysis:** By successfully cracking passwords, JtR provides tangible data on the effectiveness of an organization's password policy and the common weaknesses exploited by attackers.

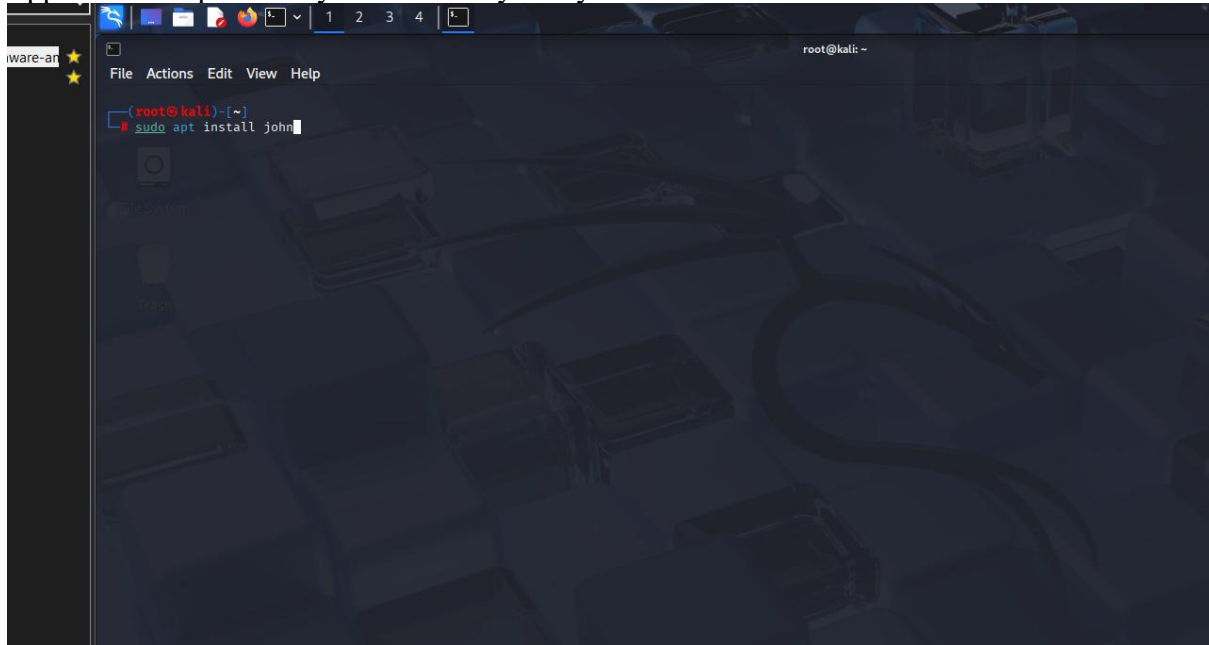
1.5. The Origin of the Name

The name "**John the Ripper**" is a direct, albeit slightly altered, reference to the infamous, unidentified serial killer **Jack the Ripper**, who terrorized London in 1888 [8]. The name is a metaphorical nod to the tool's function: just as the historical figure "ripped" through his victims, the software "rips" or tears through password hashes, systematically breaking them down to reveal the plaintext password. This choice of name is common in the security world,

where tools are often given evocative or aggressive names to reflect their powerful capabilities.

Installation

First, we have to install John the ripper in the system and it will start processing if john the ripper was not previously available in your system



2. Types of Attacks in John the Ripper

John the Ripper employs a variety of cracking modes, each designed to be effective against different types of passwords and in different scenarios. The tool typically runs these modes in a specific order, starting with the fastest and most likely to succeed, before moving to the more computationally intensive methods.

2.1. Wordlist Attack

Explanation

A **Wordlist Attack**, also known as a **Dictionary Attack**, is a method where JtR attempts to crack a password hash by comparing it against a predefined list of words, phrases, and common passwords contained in a file called a **wordlist**. This attack is based on the statistical reality that a large percentage of users choose simple, common, or dictionary-based passwords.

JtR significantly enhances this attack by applying **word mangling rules** (or "rules") to every word in the list. These rules modify the base word by adding numbers, symbols, capitalizing letters, or performing other common password variations (e.g., changing "password" to "P@ssw0rd1!").

Effectiveness and Use Cases

- **Effectiveness:** Highly effective against weak passwords, especially those based on dictionary words, names, or common patterns.
- **Situations:** This is the second mode JtR typically runs (after Single Crack). It is essential when auditing a large set of users, as a small percentage will inevitably use easily guessable passwords. The rockyou.txt wordlist, a massive collection of real-world leaked passwords, is a popular choice for this attack [11].
- **Real-World Use Cases:** Auditing a corporate network's password strength; cracking hashes recovered from a data breach to understand the scope of weak credentials.

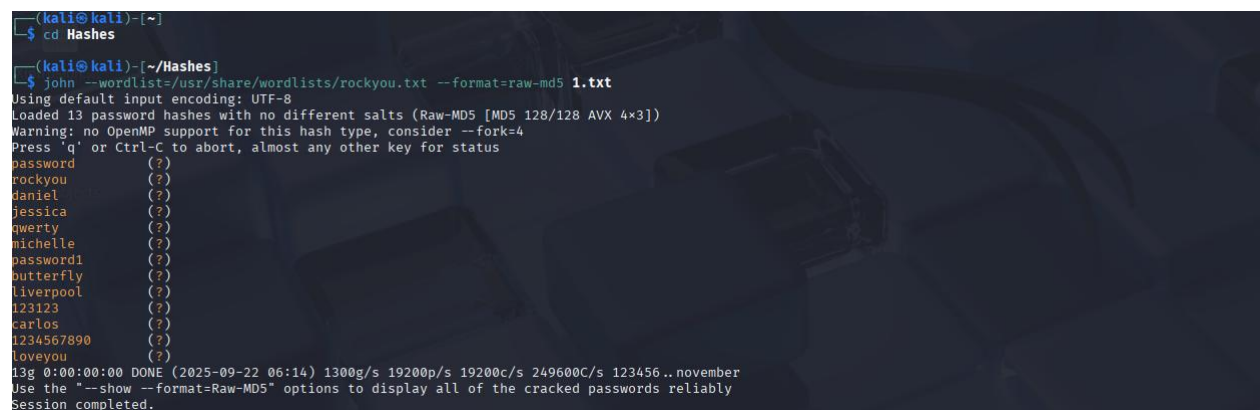
Example Command

To run a wordlist attack using a custom wordlist and applying the default mangling rules:

```
john --wordlist=custom_wordlist.txt --rules password_file.hash
```

Practical Example

Cracking MD5 Hash:



```
(kali@kali)-[~]
$ cd Hashes
(kali@kali)-[~/Hashes]
$ john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5 1.txt
Using default input encoding: UTF-8
Loaded 13 password hashes with no different salts (Raw-MD5 [MD5 128/128 AVX 4x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
password      (?)
rockyou       (?)
daniel        (?)
jessica       (?)
qwerty        (?)
michelle      (?)
password1     (?)
butterfly     (?)
liverpool     (?)
123123        (?)
carlos        (?)
1234567890    (?)
loveyou       (?)
13g 0:00:00:00 DONE (2025-09-22 06:14) 1300g/s 19200p/s 19200c/s 249600C/s 123456..november
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

MD5 (Message Digest 5)

- Fast, old cryptographic hash (128-bit output).
- Designed for integrity, not password storage.
- Fast = desirable for attackers: many candidate passwords per second on modern hardware.

- Unsalted MD5 is trivial to attack with modern GPUs and large wordlists. Salted MD5 (per-user random salt) prevents precomputed-table attacks (rainbow tables) but does *not* make the underlying function slow.

From identifying file 1.txt

Command: john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5 1.txt

Hash type: MD5

Password loaded: 13

2.2. Brute Force Attack (Incremental Mode)

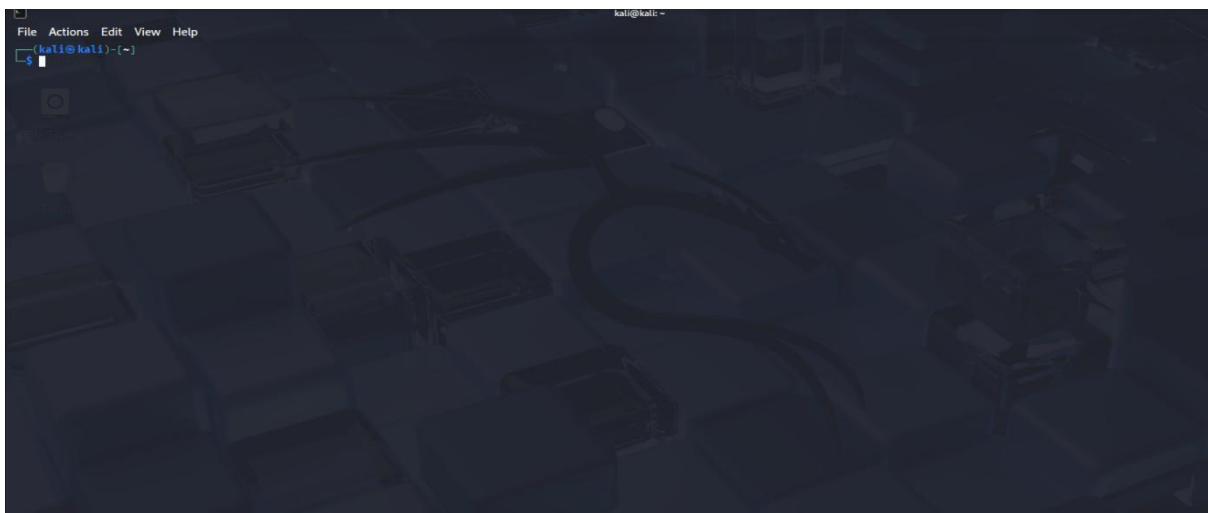
Explanation

A **Brute Force Attack** is the most exhaustive and computationally intensive cracking method. In JtR, this is implemented via the **Incremental Mode**. Brute force works by systematically trying **every possible combination** of characters (letters, numbers, symbols) for a given password length and character set.

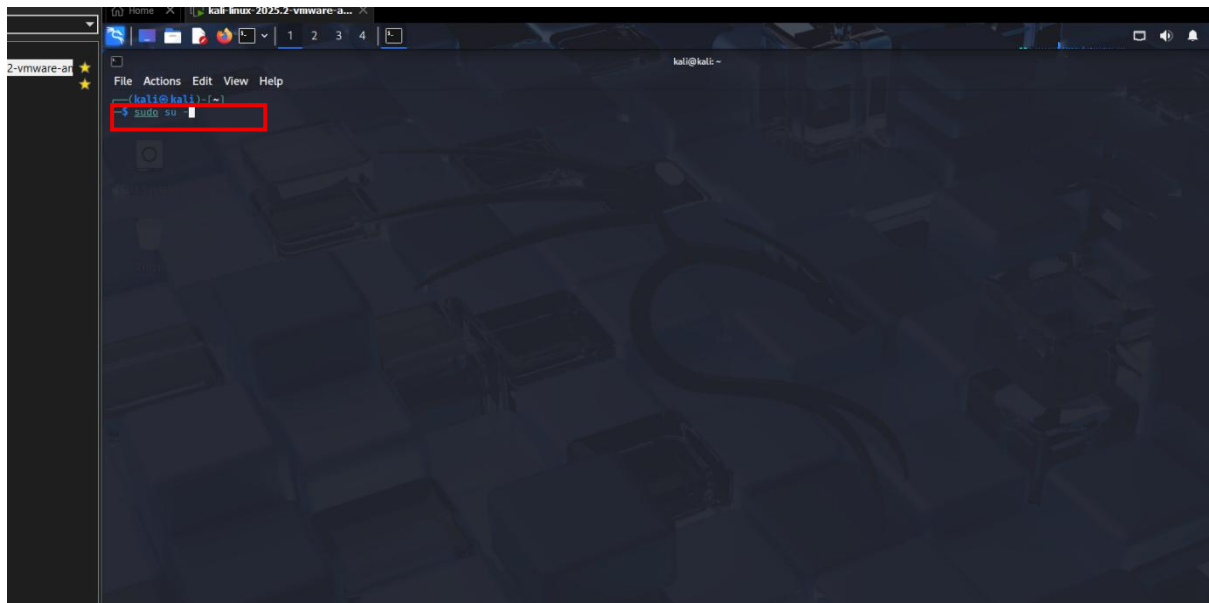
JtR's Incremental Mode is highly optimized. Instead of a simple sequential check, it uses **trigraph frequencies**—statistical data on which three-character sequences are most common in real-world passwords—to prioritize the most likely combinations first. This ensures that if the cracking session is interrupted, the most common passwords within the search space have already been checked.

We have to start with the signing in to super user.

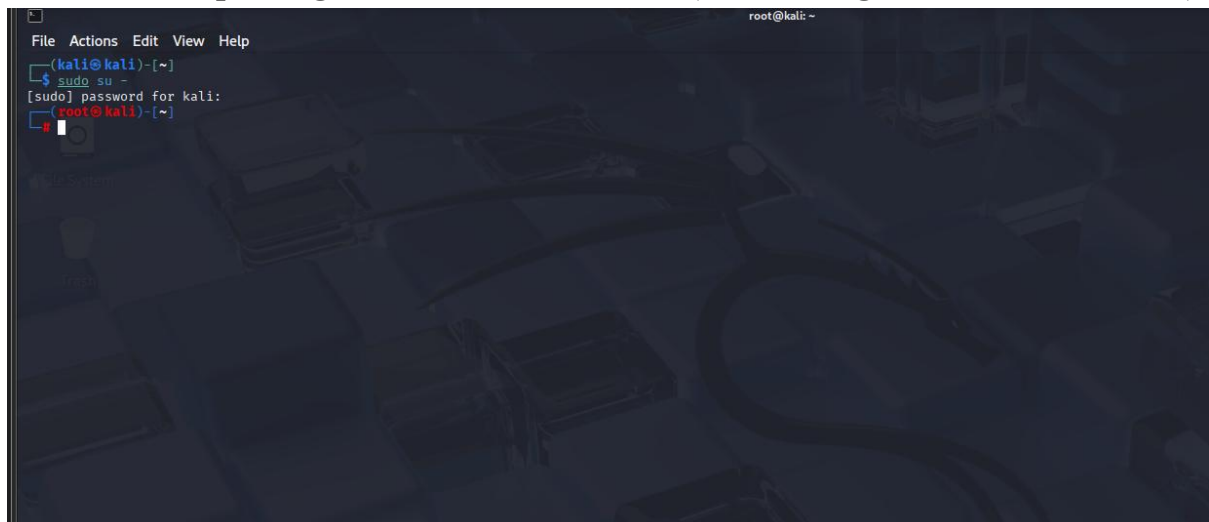
To ensure that if we are in the superuser mode or not we have to check if the Writing at the start is blue or not. If it is blue it means we are in the normal mode. Here we have to write the command.



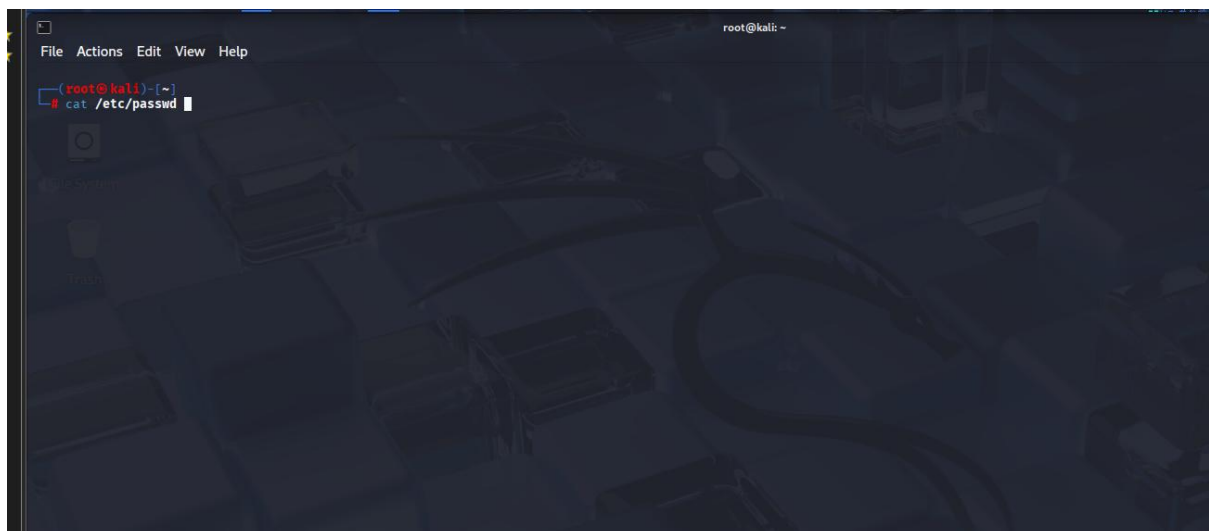
Command to enter privilege mode in the root user.



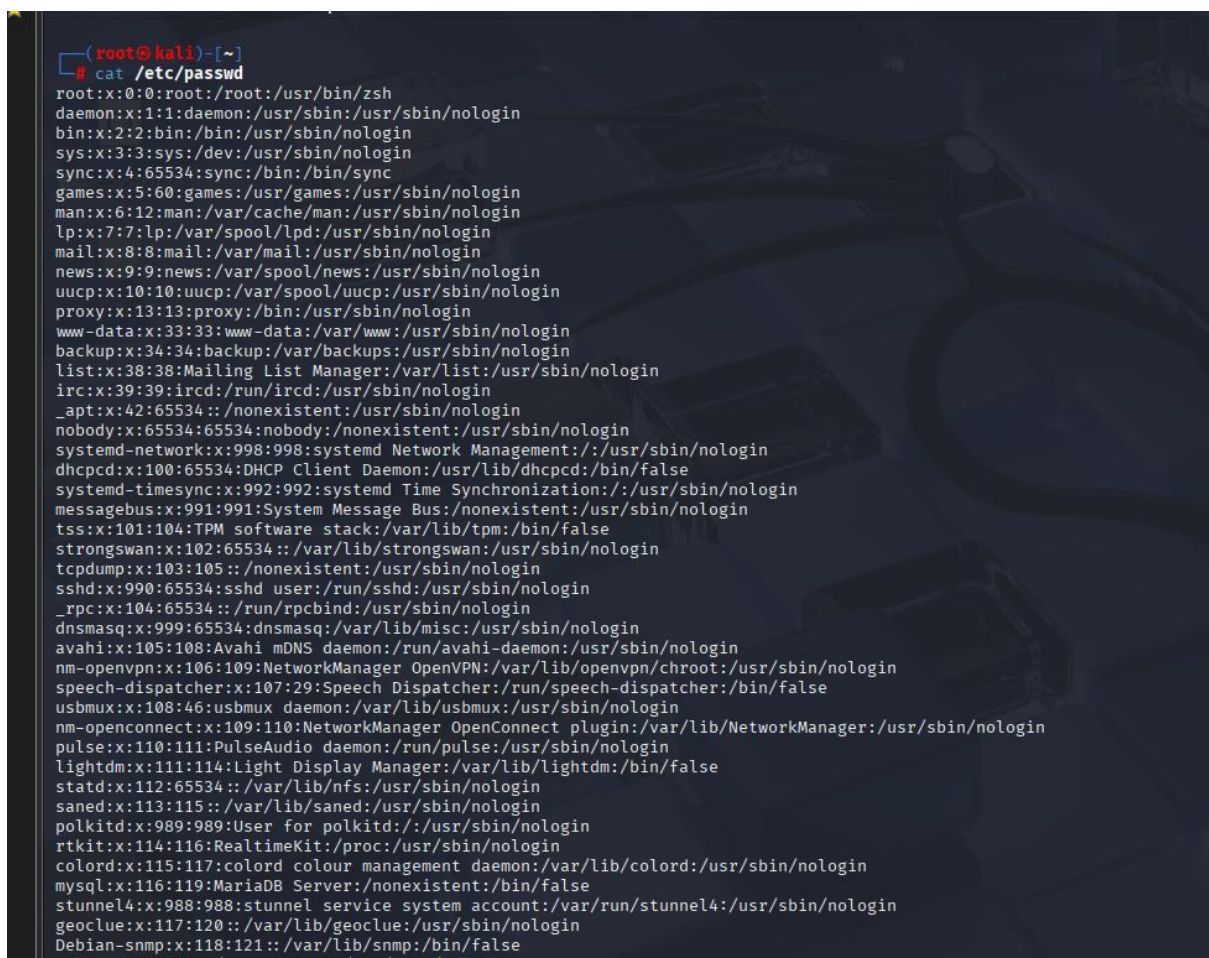
Here we have privilege access in the root user (The writing will converted to red)



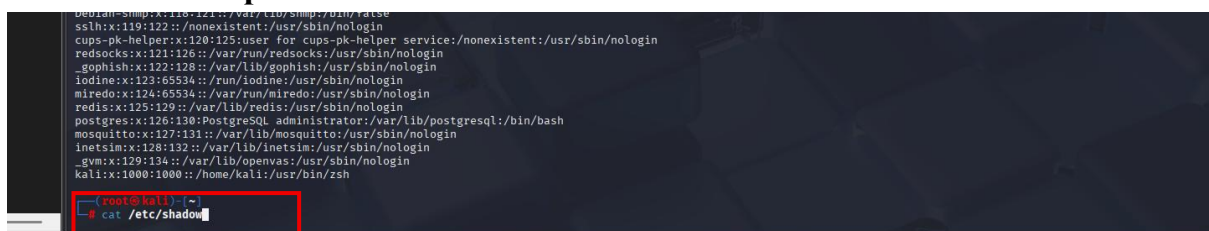
Now we have to check for the users and accounts connected with our system for which we have to enter into the category section in Kali



Now we will see all the accounts associated with our PC.



Now if we want password of each account so we have to enter Command



Here we can see a list of password of the accounts associated with our account and they all will be in Hash format no one can directly read them.

```
kali:x:1000:1000::/home/kali:/usr/bin/zsh

(root@kali)-[~]
# cat /etc/shadow
root:*:20237:0:99999:7:::
daemon:*:20237:0:99999:7:::
bin:*:20237:0:99999:7:::
sys:*:20237:0:99999:7:::
sync:*:20237:0:99999:7:::
games:*:20237:0:99999:7:::
man:*:20237:0:99999:7:::
lp:*:20237:0:99999:7:::
mail:*:20237:0:99999:7:::
news:*:20237:0:99999:7:::
uucp:*:20237:0:99999:7:::
proxy:*:20237:0:99999:7:::
www-data:*:20237:0:99999:7:::
backup:*:20237:0:99999:7:::
list:*:20237:0:99999:7:::
irc:*:20237:0:99999:7:::
_apt:*:20237:0:99999:7:::
nobody:*:20237:0:99999:7:::
systemd-network:!:20237:!:1:
dhcpcd:!:20237:!:1:
systemd-timesync:!:20237:!:1:
messagebus:!:20237:!:1:
tss:!:20237:!:1:
strongswan:!:20237:!:1:
tcpdump:!:20237:!:1:
sshd:!:20237:!:1:
_rpc:!:20237:!:1:
dnsmasq:!:20237:!:1:
avahi:!:20237:!:1:
nm-openvpn:!:20237:!:1:
speech-dispatcher:!:20237:!:1:
usbmux:!:20237:!:1:
nm-openconnect:!:20237:!:1:
pulse:!:20237:!:1:
lightdm:!:20237:!:1:
```

Now we have to create a new user to test using a command

```
(root@kali)-[~]
# adduser EhtashamJalil
new password:
Retype new password:
passwd: password updated successfully
Changing the user information for EhtashamJalil
Enter the new value, or press ENTER for the default
  Full Name []: Ehtasham Jalil
  Room Number []: 20
  Work Phone []: 1234567
  Home Phone []: 9876543
  Other []: 9877866
Is the information correct? [Y/n] y

(root@kali)-[~]
#
```

Now we have to create another account same as previous one

```
(root@kali)-[~]
# adduser AbdulRahman
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for AbdulRahman
Enter the new value, or press ENTER for the default
  Full Name []: AbdulRahman Awan
  Room Number []: 26
  Work Phone []: 098765
  Home Phone []: 67890
  Other []: 875435
Is the information correct? [Y/n] y

(root@kali)-[~]
#
```

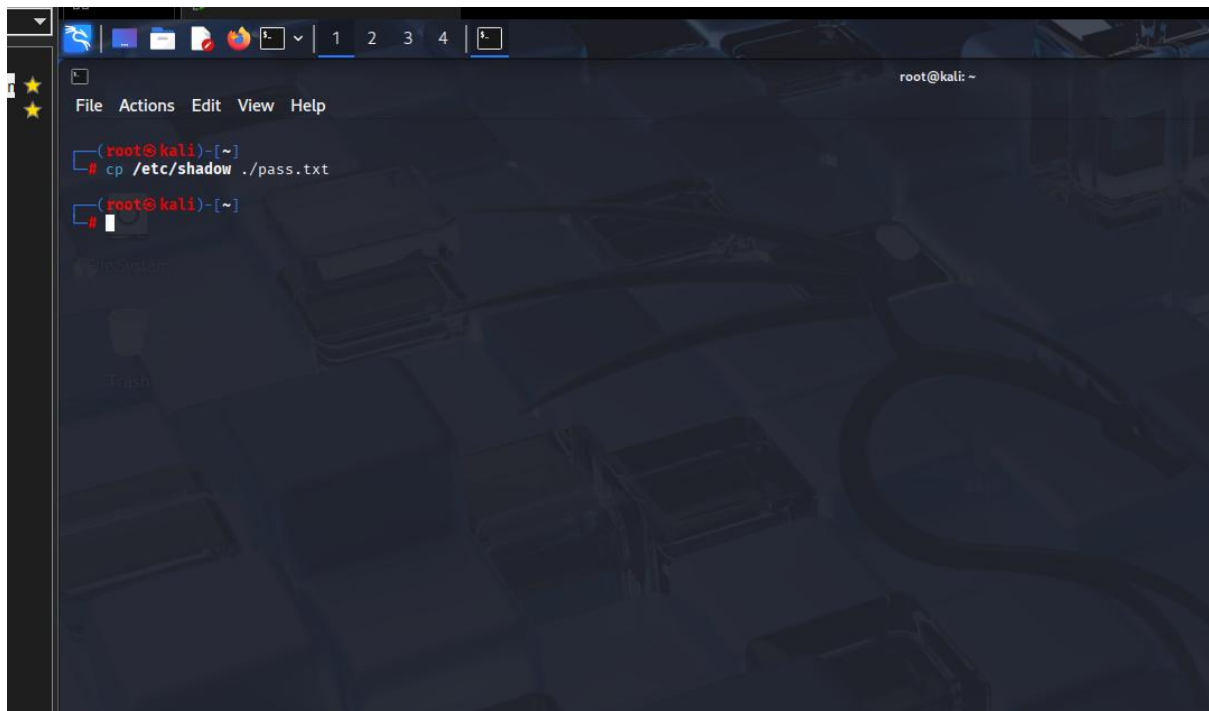
Now we want to check the account that we have created with the command

```
(root@kali)-[~]
# adduser AbdulRahman
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for AbdulRahman
Enter the new value, or press ENTER for the default
  Full Name []: AbdulRahman Awan
  Room Number []: 26
  Work Phone []: 098765
  Home Phone []: 67890
  Other []: 875435
Is the information correct? [Y/n] y
(root@kali)-[~]
# cat /etc/shadow
```

Here are the accounts that we have created and there are the Hashes that represents the Password of the account

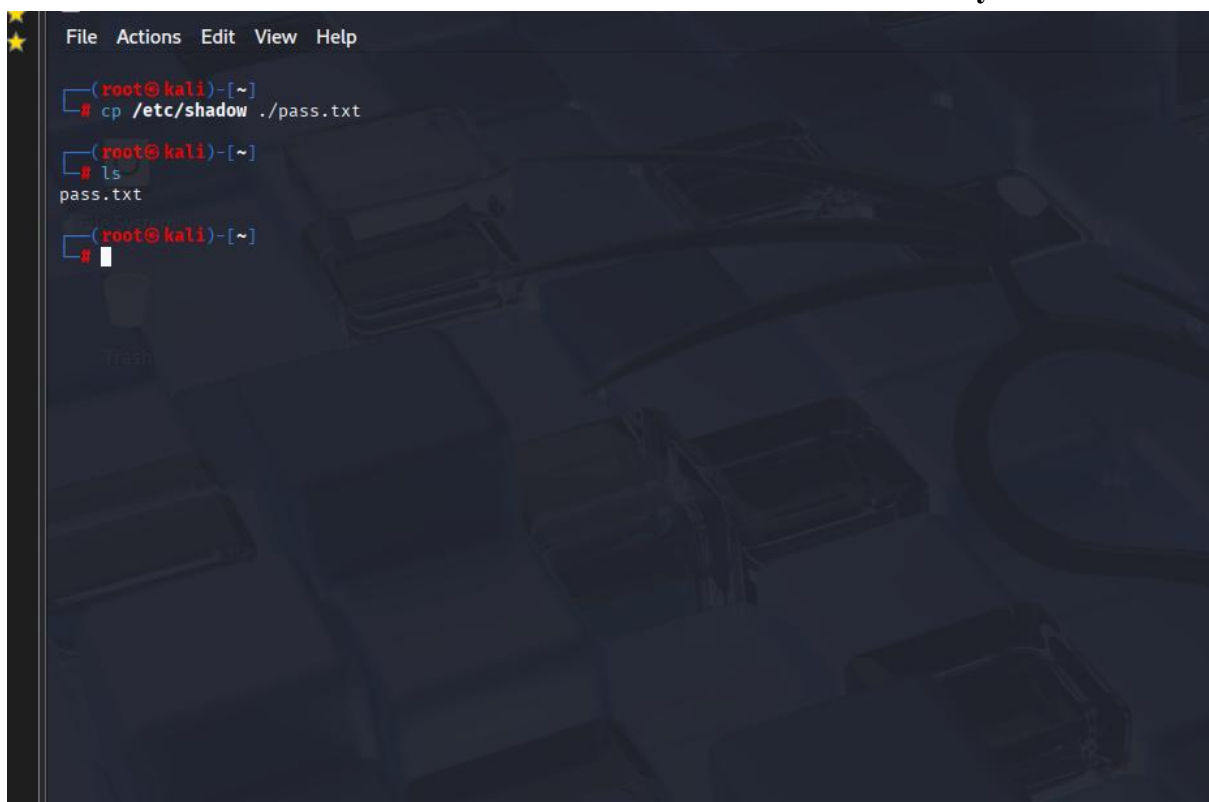
```
sshd:!*:20237:!!!!:
_rpc:!:20237:!!!!:
dnsmasq:!:20237:!!!!:
avahi:!:20237:!!!!:
nm-openvpn:!:20237:!!!!:
speech-dispatcher:!:20237:!!!!:
usbmux:!:20237:!!!!:
nm-openconnect:!:20237:!!!!:
pulse:!:20237:!!!!:
lightdm:!:20237:!!!!:
statd:!:20237:!!!!:
saned:!:20237:!!!!:
polkitd:!*:20237:!!!!:
rtkit:!:20237:!!!!:
colord:!:20237:!!!!:
mysql:!:20237:!!!!:
stunnel4:!*:20237:!!!!:
geoclue:!:20237:!!!!:
Debian-snmpp:!:20237:!!!!:
sshd:!:20237:!!!!:
cups-pk-helper:!:20237:!!!!:
redsocks:!:20237:!!!!:
_gophish:!:20237:!!!!:
iodine:!:20237:!!!!:
miredo:!:20237:!!!!:
redis:!:20237:!!!!:
postgres:!:20237:!!!!:
mosquitto:!:20237:!!!!:
inetsim:!:20237:!!!!:
_gvm:!:20237:!!!!:
kali:$y$j9T$XumnlVnY17ZEpt3859S3l1$Lfy7d0950sE9jAESSK2FMouUq9zMR42hAqZqaBo285C:20237:0:99999:7:::
Ehtasham:$y$j9T$a/VhJBGdMJtCCsE9gISLv/$mRw1j1LTtjfxnPpkyb0EAX/fFhV6AF1/Lm6U9IPnVu9:20422:0:99999:7:::
AbdulRahman:$y$j9T$KZ99XeOQ66uG9Y4FdPLiv0$9LCfThRzQaGkDPTxfzq3BW0z44oPG92cNcVFMvBpp6B:20422:0:99999:7:::
```

Here we have created a copy of “/etc/shadow”

A terminal window with a dark background and a cityscape wallpaper. The window title is 'root@kali: ~'. The menu bar shows 'File', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the command `cp /etc/shadow ./pass.txt` being executed. The prompt is `(root@kali)-[~]`.

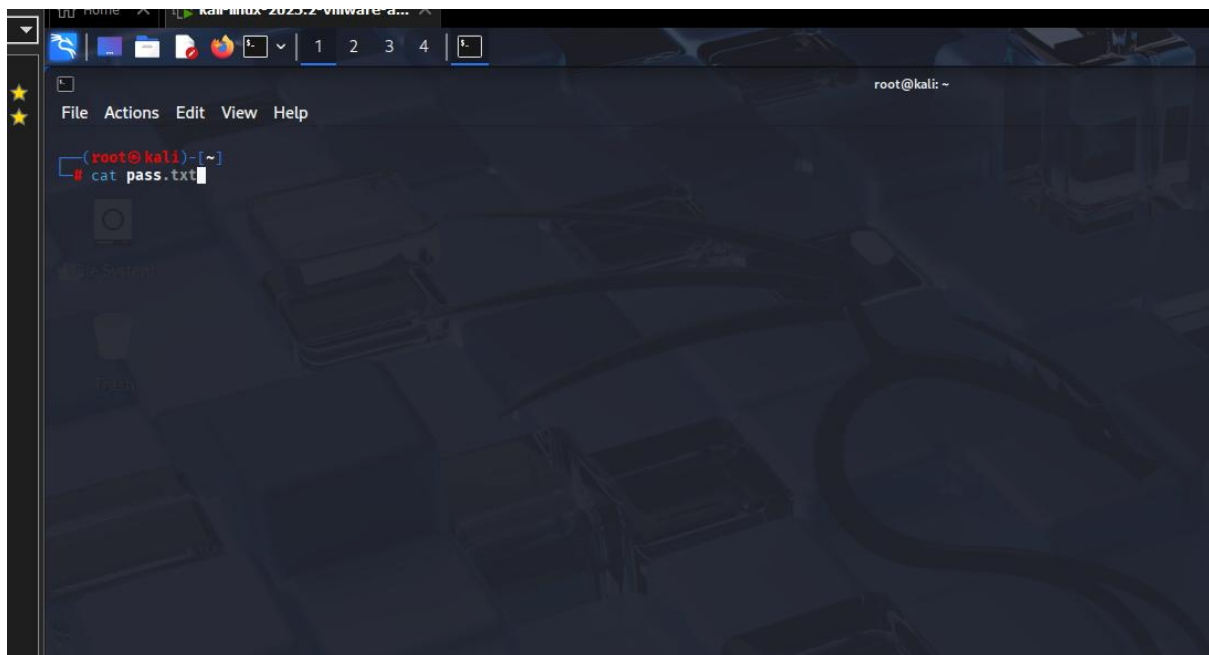
```
(root@kali)-[~]  
# cp /etc/shadow ./pass.txt  
#
```

Here we use ls command to locate our txt file that we have recently created

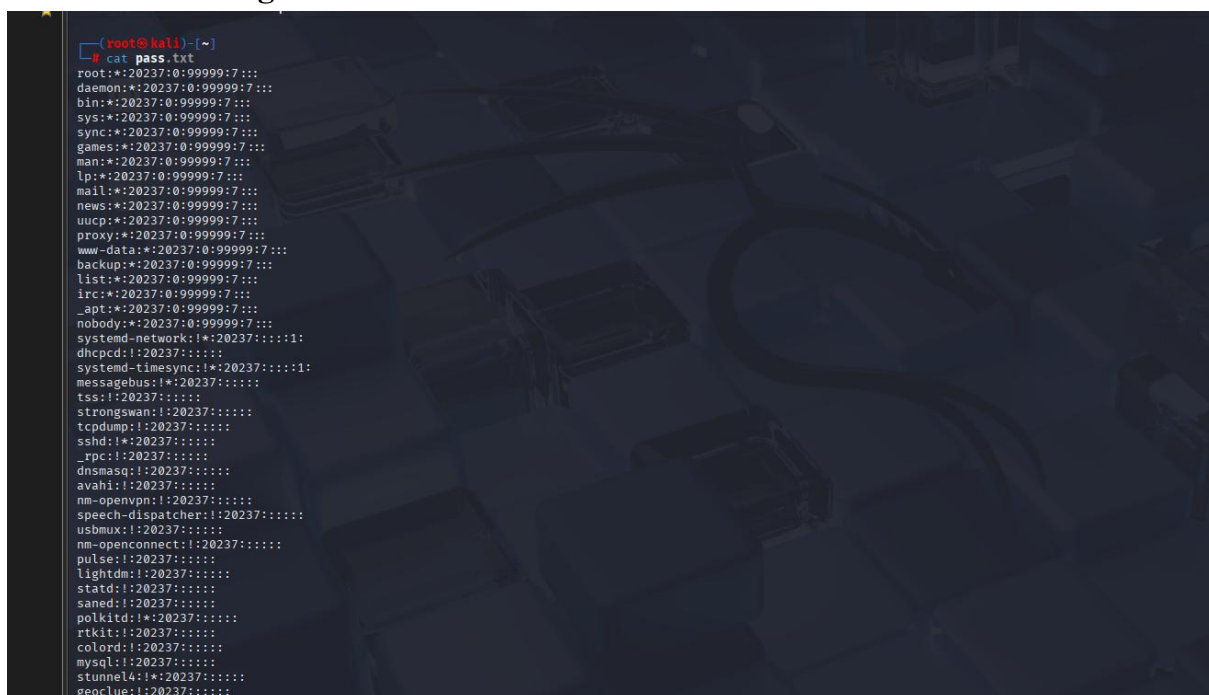
A terminal window with a dark background and a cityscape wallpaper. The window title is 'root@kali: ~'. The menu bar shows 'File', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the command `ls` being executed. The prompt is `(root@kali)-[~]`. The output of the command is `pass.txt`.

```
(root@kali)-[~]  
# cp /etc/shadow ./pass.txt  
#  
(root@kali)-[~]  
# ls  
pass.txt  
#
```

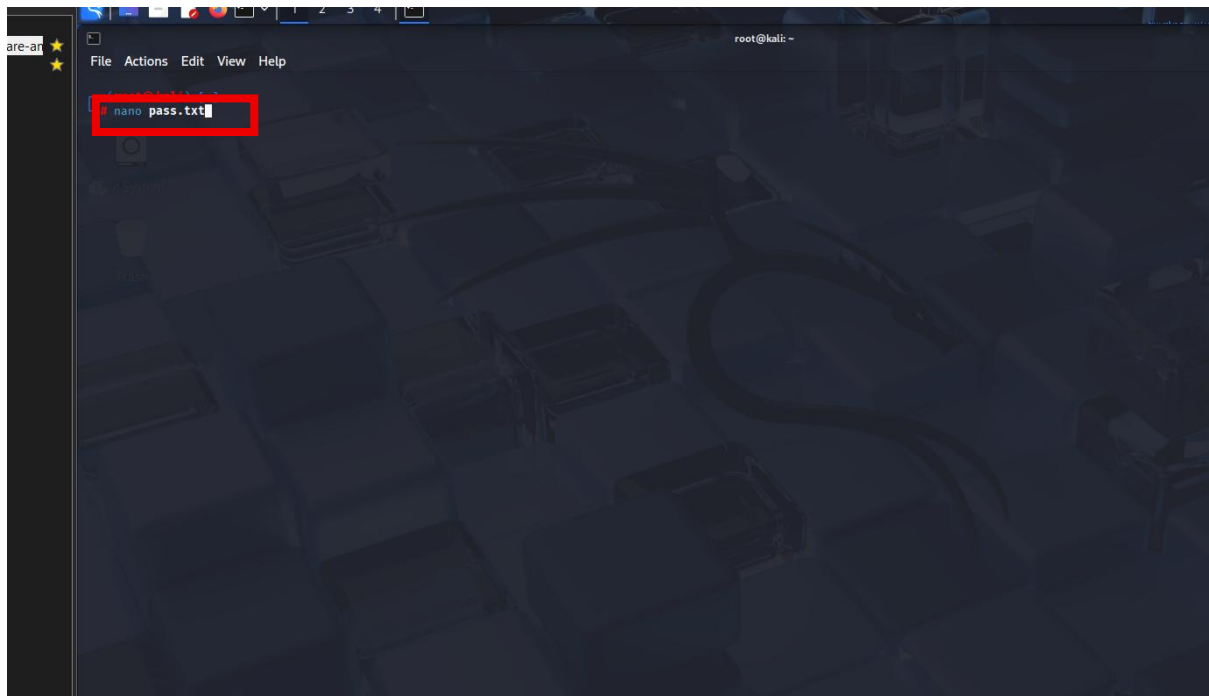
Now if we need the content in the txt file we have to write this command



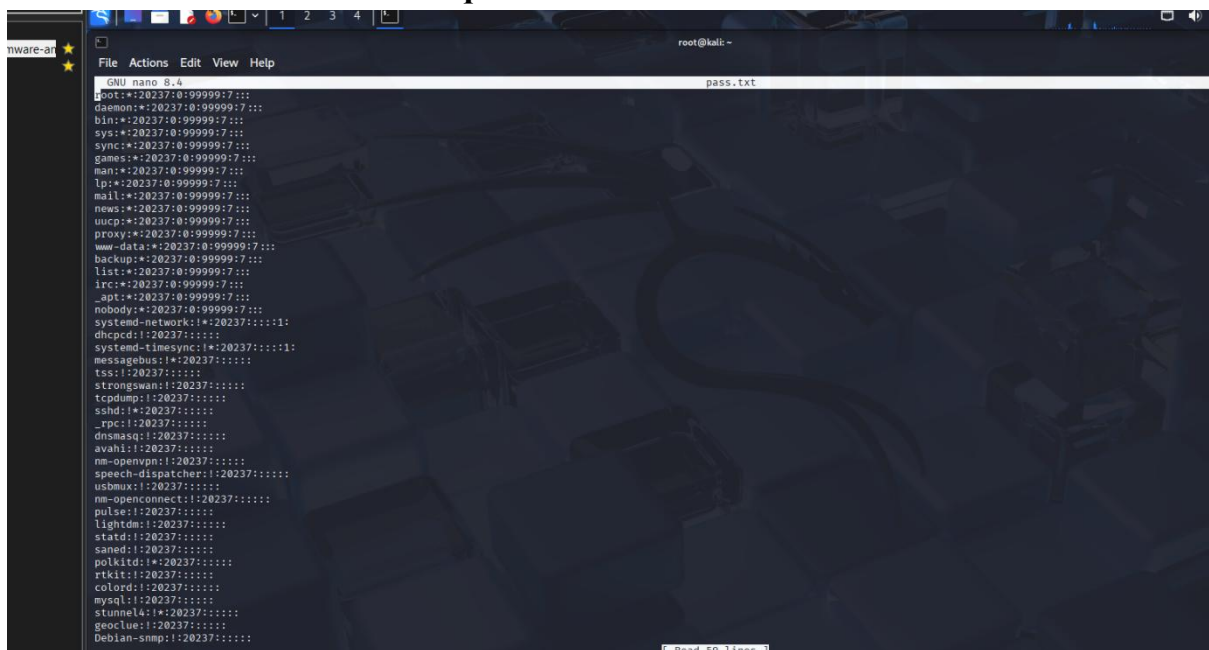
Like this we will get all the content in the txt file



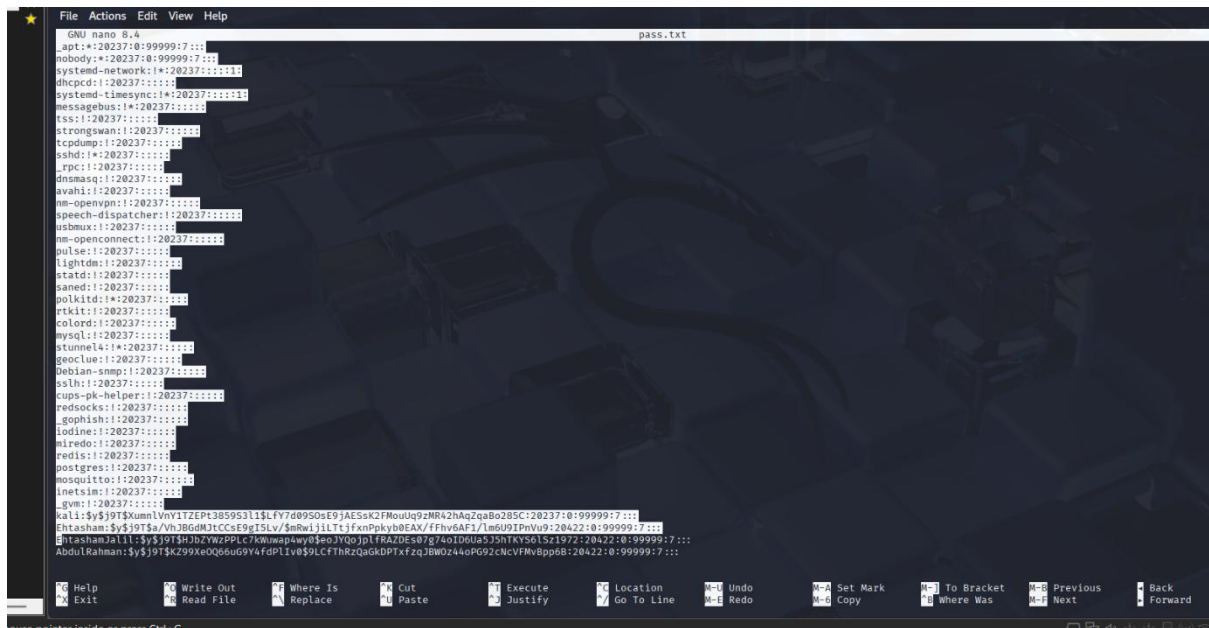
Now we don't need any extra thing in the txt file we only need the two accounts that we have created. So for that we have to write a command



Here the txt editor is opened and we can simply modify or remove the thing in the file that we are not such required



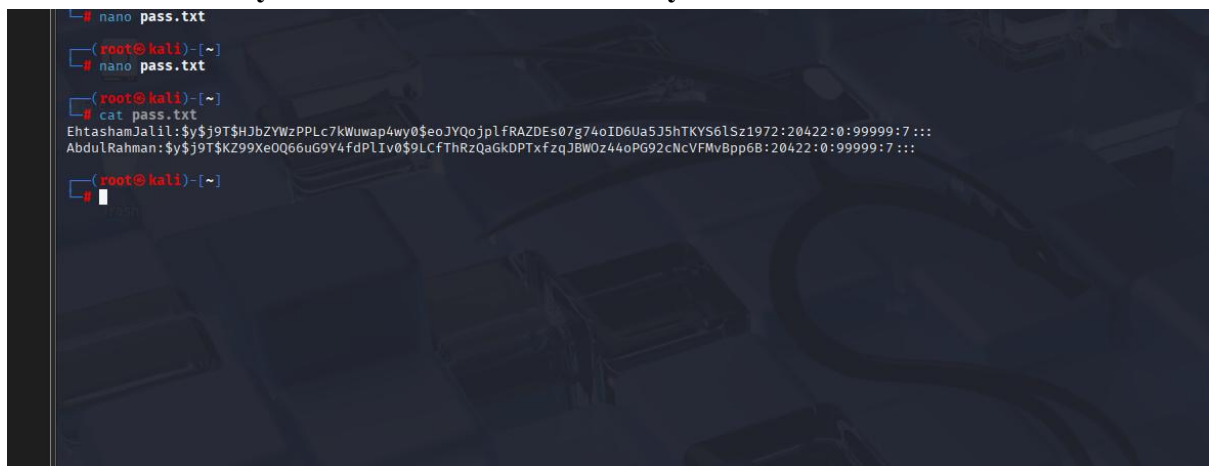
Press shift and down arrow to drag down and to select the text that we don't need and after that we have to press ctrl K to remove the text



```
GNU nano 8.4 pass.txt
_apt:*:20237:0:99999:7:::
nobody:*:20237:0:99999:7:::
systemd-networkd:*:20237:1:5::1
dhcpcd:*:20237:1:5::1
systemd-timesyncd:*:20237:1:5::1
messagebus:*:20237:1:5::1
sssd:*:20237:1:5::1
strongswan:*:20237:1:5::1
tcpdump:*:20237:1:5::1
sshd:*:20237:1:5::1
rpc:*:20237:1:5::1
dnsmasq:*:20237:1:5::1
avahi:*:20237:1:5::1
nm-openvpn:*:20237:1:5::1
speech-dispatcher:*:20237:1:5::1
usbmux:*:20237:1:5::1
nm-openconnect:*:20237:1:5::1
pulse:*:20237:1:5::1
lightdm:*:20237:1:5::1
statd:*:20237:1:5::1
sane:*:20237:1:5::1
polkitd:*:20237:1:5::1
rtkit:*:20237:1:5::1
colord:*:20237:1:5::1
mysql:*:20237:1:5::1
stunnel4:*:20237:1:5::1
geoclue:*:20237:1:5::1
Debian-snmpp:*:20237:1:5::1
sshd:*:20237:1:5::1
cups-pk-helper:*:20237:1:5::1
redsocks:*:20237:1:5::1
gophish:*:20237:1:5::1
iodine:*:20237:1:5::1
miredo:*:20237:1:5::1
redis:*:20237:1:5::1
postgres:*:20237:1:5::1
mosquitto:*:20237:1:5::1
gvm:*:20237:1:5::1
kali:$y$j9T$XumhVny1ZEPt38595311$LFY7d09S0sE9jAEssK2FMouUq9zMR42hAqZqaBo2B5C*20237:0:99999:7:::
EhtashamJalil:$y$j9T$HJbZYWzPPLc7kWuwap4wy0$eoJYQoJpLfRAZDEs07g74oID6Ua5J5hTKYS6LSz1972:20422:0:99999:7:::
AbduLRahman:$y$j9T$KZ99XeoQ66uG9Y4fdPLiv0$9LCfThRzQaGkDPTxfzqJBW0z44oPG92cNcVFMvBpp6B:20422:0:99999:7:::
```

Then we have to press ctrl O and simply press enter to save the file and then ctrl X to exit from it

Now we have only two accounts that we really need to work on



```
nano pass.txt
root@kali:~# nano pass.txt
root@kali:~# cat pass.txt
EhtashamJalil:$y$j9T$HJbZYWzPPLc7kWuwap4wy0$eoJYQoJpLfRAZDEs07g74oID6Ua5J5hTKYS6LSz1972:20422:0:99999:7:::
AbduLRahman:$y$j9T$KZ99XeoQ66uG9Y4fdPLiv0$9LCfThRzQaGkDPTxfzqJBW0z44oPG92cNcVFMvBpp6B:20422:0:99999:7:::
```

Now we need to crack the password of the accounts that we have created using the command


```
(root@kali)-[~]
# cat pass.txt
EhtashamJalil:$y$j9T$HJbZYWzPPLc7kWuwap4wy0$eoJYQojp1FRAZDEs07g74oID6Ua5J5hTKYS6lSz1972:20422:0:99999:7:::
AbdulRahman:$y$j9T$KZ99XeOQ66uG9Y4fdPLIv0$9LCfThRzQaGkDPTxfzqJBW0z44oPG92cNcVFMvBpp6B:20422:0:99999:7:::

(root@kali)-[~]
# john -format=crypt pass.txt
```



Here we see it starts working on it

```
AbdulRahman:$y$j9T$KZ99XeOQ66uG9Y4fdPLIv0$9LCfThRzQaGkDPTxfzqJBW0z44oPG92cNcVFMvBpp6B:20422:0:99999:7:::

(root@kali)-[~]
# john -format=crypt pass.txt
Created directory: /root/.john
Using default input encoding: UTF-8
Loaded 2 password hashes with 2 different salts (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status

```

Here we have got the password of the account that we have created

```
(root@kali)-[~]
# john -format=crypt pass.txt
Created directory: /root/.john
Using default input encoding: UTF-8
Loaded 2 password hashes with 2 different salts (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Warning: Only 4 candidates buffered for the current salt, minimum 96 needed for performance.
Proceeding with wordlist:/usr/share/dict/passwd.lst
Lucky      (AbdulRahman)
Kali       (EhtashamJalil)
2g 0:00:03:01 DONE 2/3 (2025-11-30 17:22) 0.01104g/s 71.34p/s 94.15c/s 94.15C/s Free..Makeitso
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(root@kali)-[~]
#
```

Diagram labels:

- Passwords (points to the list of cracked passwords)
- User Accounts (points to the usernames in parentheses)

Now if we want to verify that if it really crack all the passwords in the file or not we have to write this command

```
(root@kali)-[~]
# john --show pass.txt
```

Here we go with the complete verification

```
session completed.  
  
(root@kali)-[~]  
# john --show pass.txt  
EhtashamJalil:Kali:20422:0:99999:7:::  
AbdulRahman:Lucky:20422:0:99999:7:::  
  
2 password hashes cracked, 0 left  
  
(root@kali)-[~]  
#
```

2.3. Single Crack Attack

Explanation

The **Single Crack Attack** is the fastest and most efficient mode, and it is the first mode JtR attempts by default. It is based on the premise that many users choose passwords that are closely related to their personal or account information.

In this mode, JtR extracts information from the password file, such as the **username (login name)**, the **GECOS field** (which often contains the user's full name, office number, etc.), and the **home directory name**. It then uses this extracted information as the base for a dictionary attack, applying a massive set of highly aggressive mangling rules to guess passwords like john123, JohnDoe, or password123.

What Is Single Crack Attack?

The **Single Crack Attack** is an intelligent password-guessing mode in John the Ripper. Instead of using a wordlist or brute force, John generates smart password guesses based on:

- Username
- GECOS field (user information)
- System account details
- Hybrid variations (adding numbers, capitalizing, appending patterns)

This attack imitates **real-world attacks** where users often create passwords using personal information such as:

- their name
- nickname
- birth year
- job title
- phone number

- department

Understanding the Password Hash File (hash.txt)

To perform the Single Crack Attack, we created a file named `single_Crack_Attack.txt` containing:

```
abdurrahman:$1$n/87UpPj$qd2a.PK.17R3ELeexEdA4.:1001:1001:Abdurrahman,  
CyberSec Student:/home/abdurrahman:/bin/bash
```

```
(kali@kali)-[~/Project]  
$ cat single_Crack_Attack.txt  
abdurrahman:$1$n/87UpPj$qd2a.PK.17R3ELeexEdA4.:1001:1001:Abdurrahman, CyberSec Student:/home/abdurrahman:/bin/bash
```

This file contains 3 main parts:

1. Username

```
abdurrahman
```

John uses this as a base word to generate guesses:

- abdurrahman
- Abdurrahman
- abdurrahman123
- abdurrahman007
- abdurrahman@123
- ...and hundreds of smart variations.

2. Password Hash

```
$1$n/87UpPj$qd2a.PK.17R3ELeexEdA4.
```

This is an **md5crypt** hash.

John tries to find the original plaintext by testing guesses until one matches this hash.

3. GECOS Field

```
Abdurrahman, CyberSec Student
```

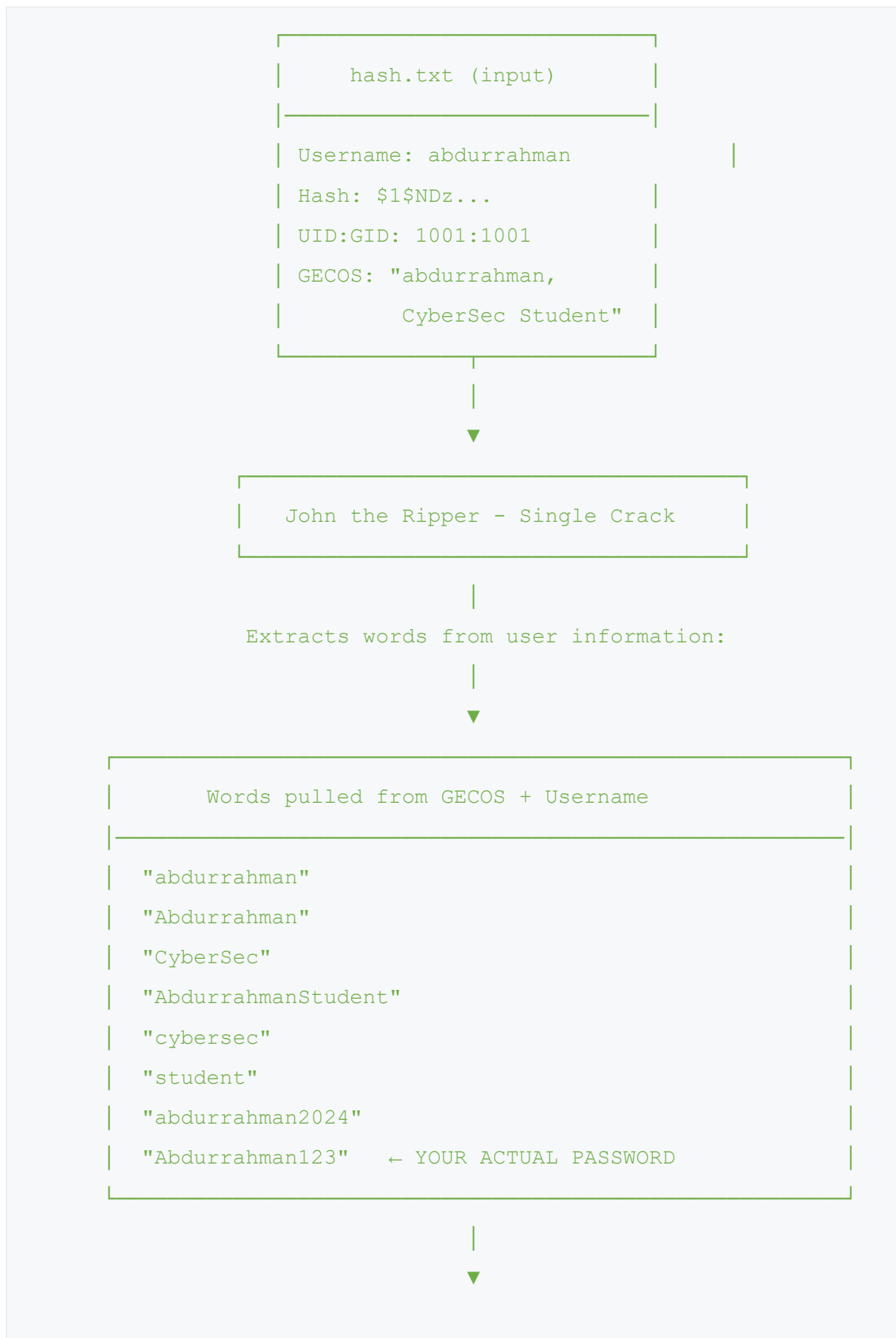
This is extra information about the user found in the `/etc/passwd` file on Linux systems.

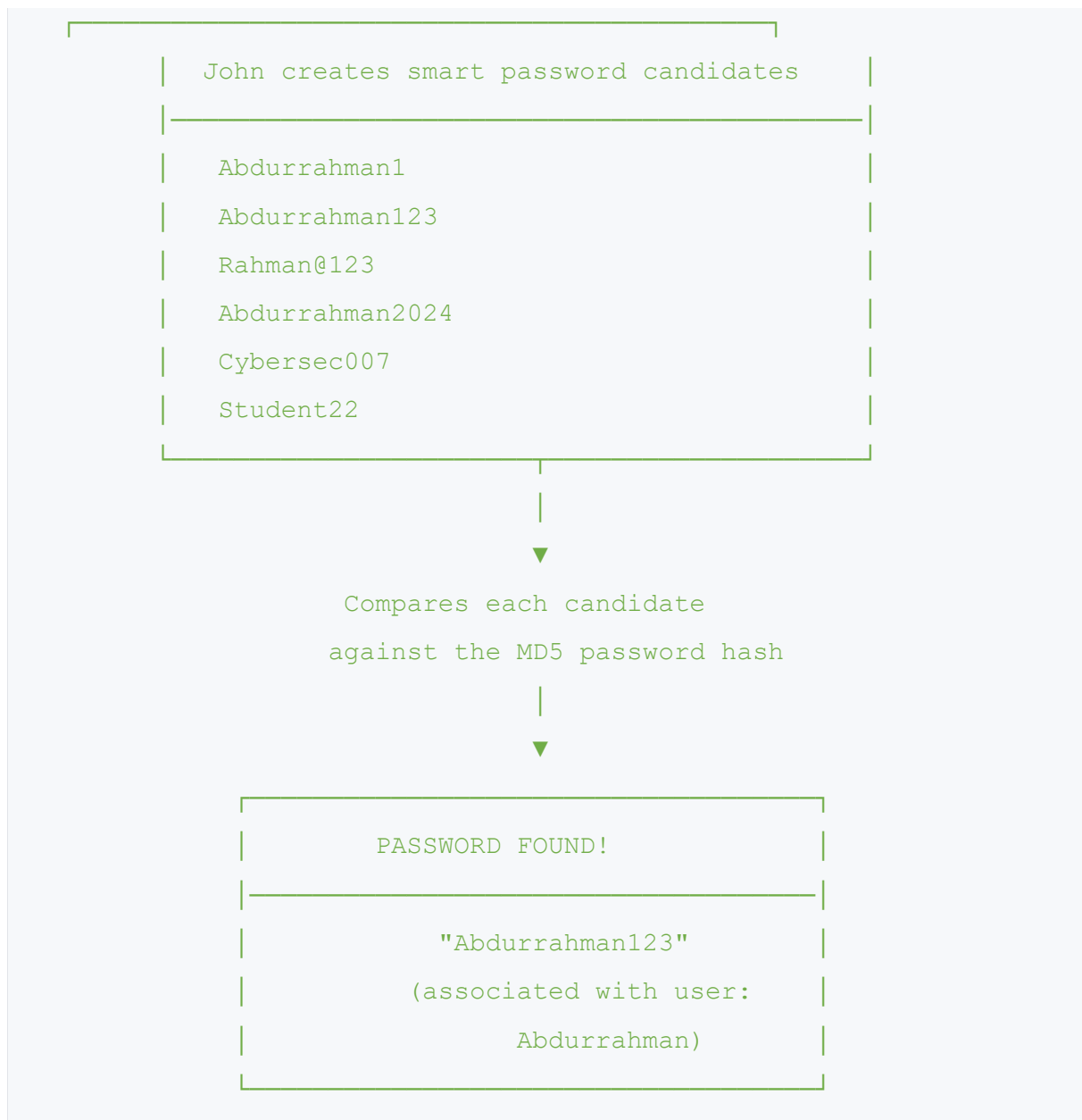
John extracts names from here:

- CyberSec
- Abdurrahman
- Student
- StudentCyberSec

- AbdurrahmanStudent
...and generates password variations such as:
- Abdurrahman123
- Student007
- CyberSec2024
- Abdurrahman@321

How John the Ripper Uses GECOS (With Diagram)





Explanation

- John reads the **username** and **GECOS field** from your hash file.
- GECOS gives personal information like:
 - Full name
 - Department
 - Occupation
- John extracts keywords like:
 - abdurrahman
 - Abdurrahman
 - CyberSec
 - Student

- John then generates **intelligent password guesses**, such as:
 - Abdurrahman123
 - Abdurrahman2024
 - Student007
- If one matches the hash → the password is cracked instantly.

This is why your demo cracked Abdurrahman123 immediately.

Performing the Single Crack Attack (Practical Execution)

Step 1 — Creating the hash file

We stored the following entry in **hash.txt**:

```
abdurrahman:$1$n/87UpPj$qd2a.PK.17R3ELeexEdA4.:1001:1001:Abdurrahman,
CyberSec Student:/home/abdurrahman:/bin/bash
```

Step 2 — Run the Single Crack Attack

Command used:

```
john --single single_Crack_Attack.txt
```

```
(kali@kali)-[~/Project]
$ john --single single_Crack_Attack.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 128/128 AVX 4x3])
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Abdurrahman123 (abdurrahman)
1g 0:00:00:00 DONE (2025-12-07 03:10) 8.333g/s 64000p/s 64000c/s 64000C/s acybersec123..student777
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Step 3 — Output and Result

John successfully cracked the password:

Abdurrahman123 (abdurrahman)

This means the original password for the user "abdurrahman" was Abdurrahman123.

```

kali@kali: ~/Project
Session Actions Edit View Help
GNU nano 8.6 single_Crack_Attack.txt
abdurrahman:$1$n/87UpPj$qd2a.PK.17R3ELeexEdA4.:1001:1001:Abdurrahman, CyberSec Student:/home/abdurrahman:/bin/bash
```

Why the Single Crack Attack Worked

Because:

- The password contained the user's own name → "Abdurrahman"
- Followed by simple numbers → "123"
- Both patterns are exactly what John expects during Single Crack mode
- GECOS field also contained "Hanan", which helped John generate variations

This mode is extremely effective against:

- Weak passwords
- Human-made password patterns
- System / user accounts stored in /etc/passwd style files

Conclusion

The **Single Crack Attack** in John the Ripper is a highly optimized and targeted attack that uses information already available in user account files.

It does not require dictionary files or brute-force strategies.

In our project practical, we successfully cracked a password using:

- md5crypt hash
- username
- GECOS field
- John's mutation rules

This demonstrates how attackers can exploit poor password habits and why organizations must enforce stronger password policies.

Effectiveness and Use Cases

- **Effectiveness:** This attack is extremely effective because it targets the most common human weakness in password creation: using easily accessible personal data. Since the information is only tested against the hash of the account it was taken from, it is incredibly fast.
- **Situations:** Ideal for initial, rapid auditing of a password file. If a password is crackable by this method, it is a critical security failure.
- **Example Use Cases:** Cracking a password that is the user's first name followed by their birth year (e.g., Sarah1995); cracking a password that is the company name with a number appended.

2.4. Mask Attack

Explanation

The **Mask Attack** is a highly targeted, pattern-based attack that sits between the Wordlist and Brute Force modes in terms of computational cost and effectiveness. It is used when the attacker has a strong suspicion about the **structure or pattern** of the password, but not the exact characters.

A **mask** is a string that defines the exact composition of the password using special placeholders. For example, if an attacker knows the password is a 4-digit PIN, they can specify a mask that only checks 4-digit combinations, drastically reducing the search space compared to a full brute force attack.

Placeholder	Character Set	Example	Meaning
<u>?l</u>	Lowercase letters (<u>a-z</u>)	<u>?l?l?l</u>	Any 3 lowercase letters
<u>?u</u>	Uppercase letters (<u>A-Z</u>)	<u>?u?u?u</u>	Any 3 uppercase letters
<u>?d</u>	Digits (<u>0-9</u>)	<u>?d?d?d?d</u>	Any 4 digits (0000 to 9999)
<u>?s</u>	Symbols	<u>?s?s?s</u>	Any 3 symbols
<u>?a</u>	All printable characters	<u>?a?a?a?a</u>	Any 4 printable characters

Time-Saving and Scenarios

- **Time-Saving:** By limiting the search space to a specific pattern (e.g., 4 digits instead of 8 random characters), the Mask Attack saves significant time compared to a full brute force attack.
- **When Used:** This method is used when intelligence suggests a specific password policy (e.g., "all passwords must end with two digits") or when a partial password has been discovered.
- **Example Scenario:** A system administrator knows that all users were forced to use a 6-character password consisting of a known 3-letter prefix (e.g., "abc") followed by three random digits. The mask would be abc?d?d?d.

Practical Example

In this project, I demonstrate how attackers use **mask attacks** in password-cracking tools such as **John the Ripper**.

A mask attack is a **targeted brute force method**, where we reduce the search space by defining a **pattern** for the password.

This makes cracking faster and more practical compared to full brute force, which tries every possible combination blindly.

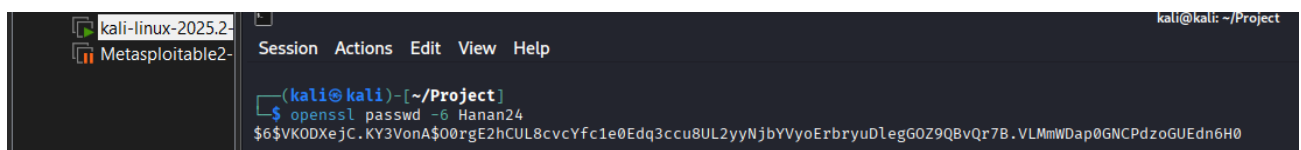
Step 1 — Creating the Hashed Password

First, I generated a password hash using OpenSSL.

Command Used:

```
openssl passwd -6 Hanan24
```

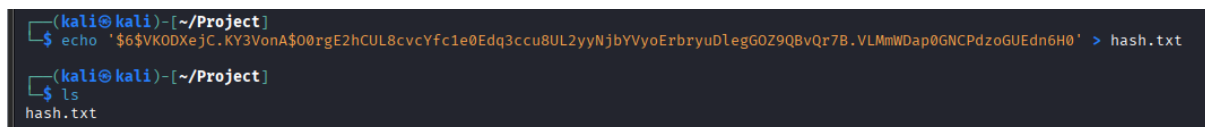
This produced a SHA512crypt hash.



```
kali@kali: ~/Project
(kali@kali)~$ openssl passwd -6 Hanan24
$6$VK0DXejC.KY3VonA$00rgE2hCUL8cvcYfc1e0Edq3ccu8UL2yyNjbYVyoErbryuDlegGOZ9QBvQr7B.VLMmWDap0GNCPdzoGUEdn6H0
```

Then I saved the hash into file named hash.txt.

```
echo '$6$VK0DXejC.KY3VonA$00rgE2hCUL8cvcYfc1e0Edq3ccu8UL2yyNjbYVyoErbryuDlegGOZ9QBvQr7B.VLMmWDap0GNCPdzoGUEdn6H0' > hash.txt
```

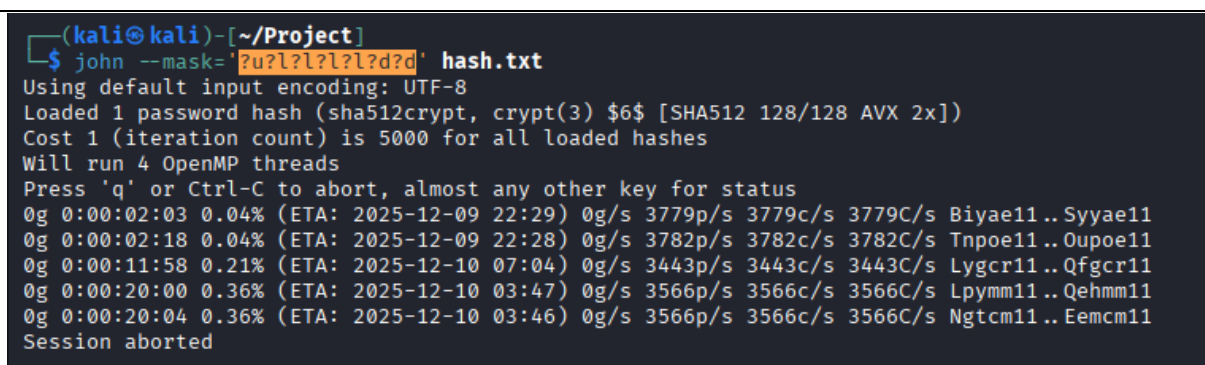


```
(kali@kali)~$ echo '$6$VK0DXejC.KY3VonA$00rgE2hCUL8cvcYfc1e0Edq3ccu8UL2yyNjbYVyoErbryuDlegGOZ9QBvQr7B.VLMmWDap0GNCPdzoGUEdn6H0' > hash.txt
(kali@kali)~$ ls
hash.txt
```

Step 2 — Running a Full Mask Attack

I attempted to crack the password using the following mask:

```
?u?l?l?l?l?d?d
```



```
(kali@kali)~$ john --mask='?u?l?l?l?l?d?d' hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:02:03 0.04% (ETA: 2025-12-09 22:29) 0g/s 3779p/s 3779c/s 3779C/s Biyae11..Syxae11
0g 0:00:02:18 0.04% (ETA: 2025-12-09 22:28) 0g/s 3782p/s 3782c/s 3782C/s Tnpoe11..Oupoe11
0g 0:00:11:58 0.21% (ETA: 2025-12-10 07:04) 0g/s 3443p/s 3443c/s 3443C/s Lygcr11..Qfgcr11
0g 0:00:20:00 0.36% (ETA: 2025-12-10 03:47) 0g/s 3566p/s 3566c/s 3566C/s Lpymm11..Qehmm11
0g 0:00:20:04 0.36% (ETA: 2025-12-10 03:46) 0g/s 3566p/s 3566c/s 3566C/s Ngtdcm11..Eemcm11
Session aborted
```

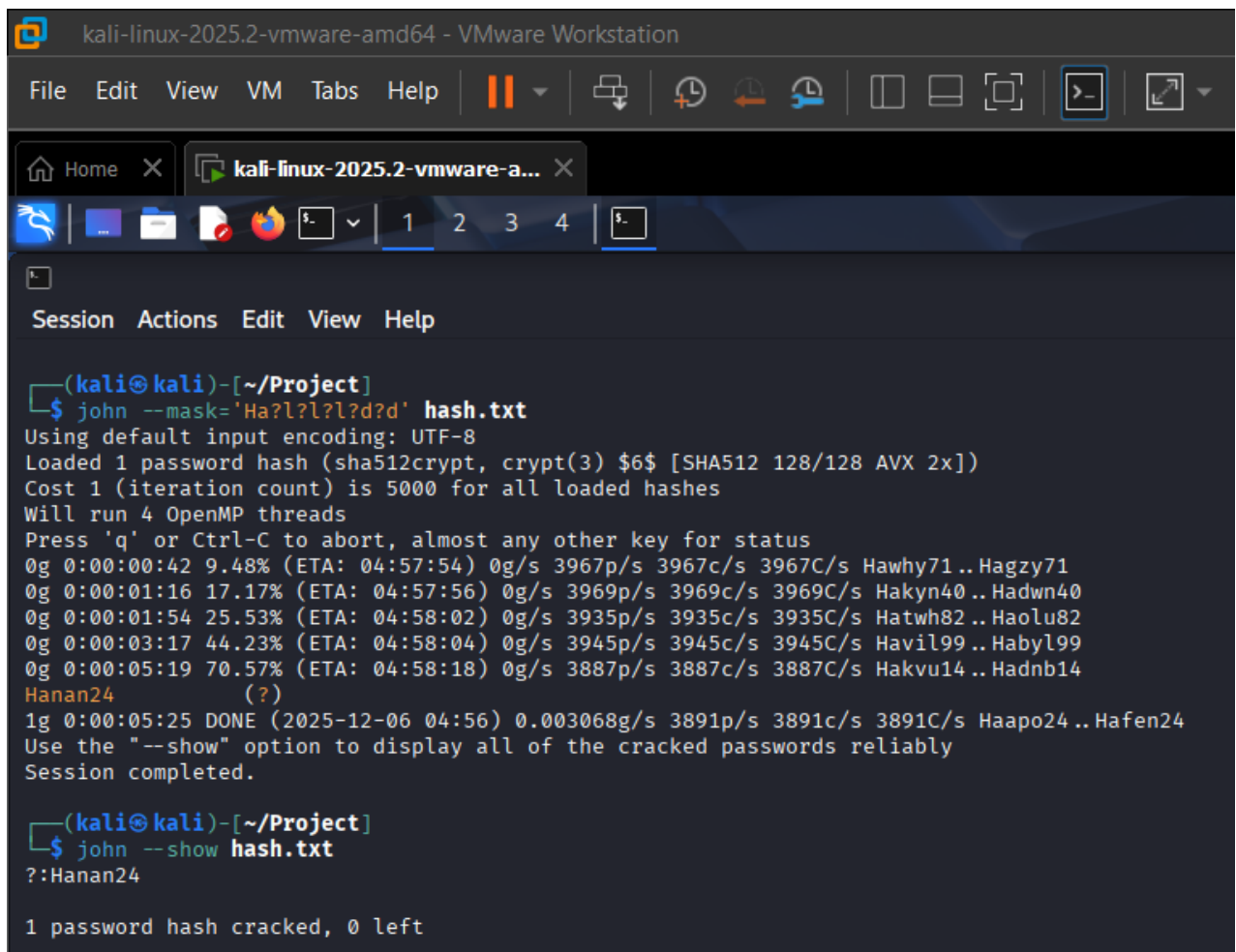
- **?u** = One uppercase letter
- **?l?l?l** = Three lowercase letters
- **?d?d** = Two digits

Another Scenario:

Optimized Mask Attack (Attacker Knows Some Characters)

“Attacker knows that the password starts with **Ha**”

Ha?l?l?l?d?d



```
(kali㉿kali)-[~/Project]
$ john --mask='Ha?l?l?l?d?d' hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:42 9.48% (ETA: 04:57:54) 0g/s 3967p/s 3967c/s 3967C/s Hawhy71..Hagzy71
0g 0:00:01:16 17.17% (ETA: 04:57:56) 0g/s 3969p/s 3969c/s 3969C/s Hakyn40..Hadwn40
0g 0:00:01:54 25.53% (ETA: 04:58:02) 0g/s 3935p/s 3935c/s 3935C/s Hatwh82..Haolu82
0g 0:00:03:17 44.23% (ETA: 04:58:04) 0g/s 3945p/s 3945c/s 3945C/s Havil99..Haby199
0g 0:00:05:19 70.57% (ETA: 04:58:18) 0g/s 3887p/s 3887c/s 3887C/s Hakvu14..Hadnb14
Hanan24 (?)
1g 0:00:05:25 DONE (2025-12-06 04:56) 0.003068g/s 3891p/s 3891c/s 3891C/s Haapo24..Hafen24
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali㉿kali)-[~/Project]
$ john --show hash.txt
?:Hanan24

1 password hash cracked, 0 left
```

- Starts with **Ha**
- Followed by **3 lowercase letters**
- Ends with **2 digits**

3. Conclusion

John the Ripper stands as a foundational and continually evolving tool in the field of cybersecurity. Its creator, Solar Designer, and the Openwall Project have provided the community with a highly optimized, versatile, and essential utility for password security auditing. By employing a strategic sequence of cracking modes—starting with the rapid, intelligence-based **Single Crack** and **Wordlist** attacks, and escalating to the targeted **Mask Attack** and the exhaustive **Incremental (Brute Force) Mode**—JtR offers a comprehensive methodology for identifying and mitigating weak credentials. The ability to simulate real-world password attacks makes JtR an invaluable asset for penetration testers, forensic analysts, and system administrators committed to maintaining robust digital security.
